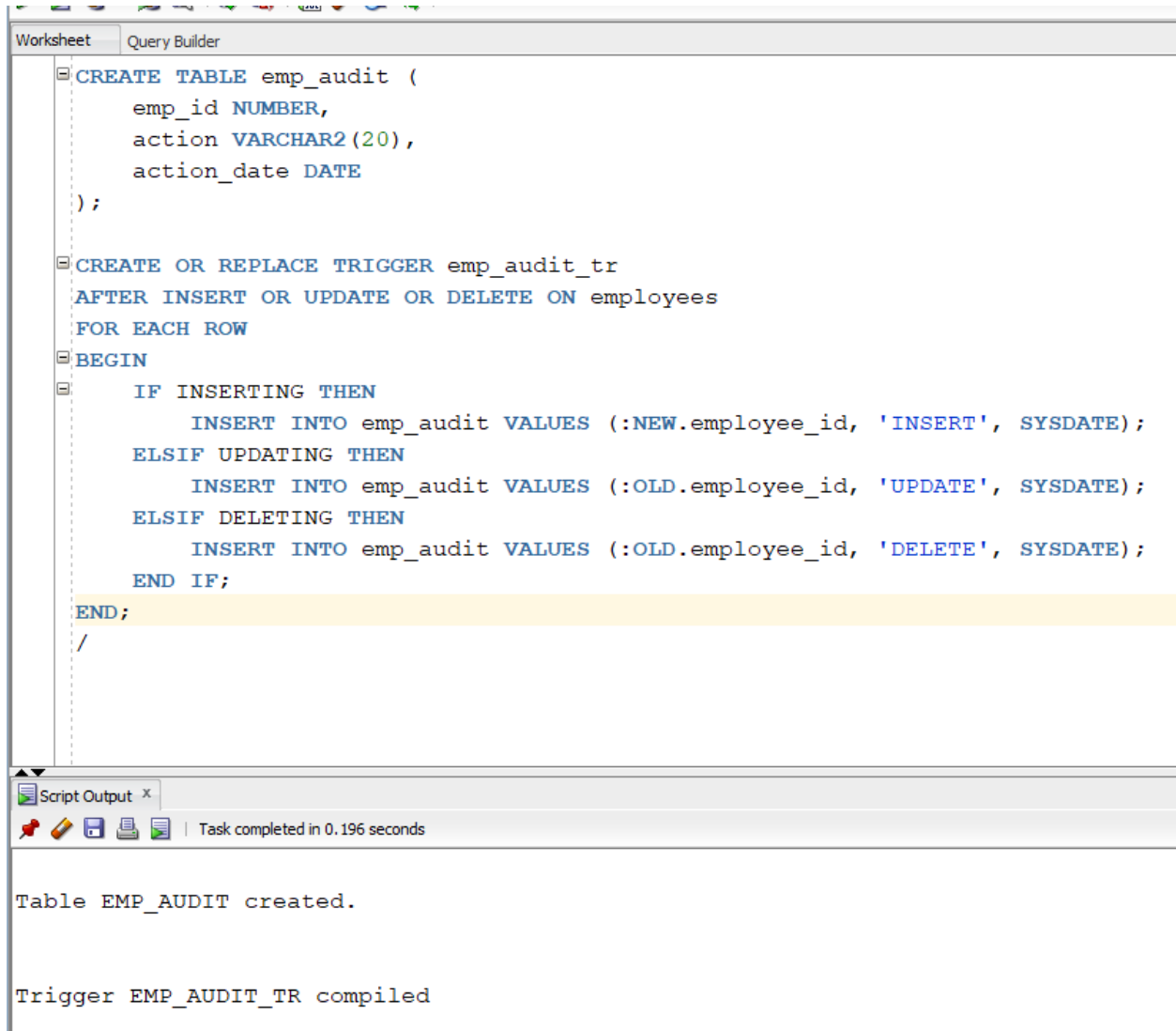


DBS LAB 8

DML Trigger Task:



The screenshot shows a database query editor window with two tabs: "Worksheet" and "Query Builder". The "Query Builder" tab is active, displaying the following SQL script:

```
CREATE TABLE emp_audit (  
    emp_id NUMBER,  
    action VARCHAR2(20),  
    action_date DATE  
);  
  
CREATE OR REPLACE TRIGGER emp_audit_tr  
AFTER INSERT OR UPDATE OR DELETE ON employees  
FOR EACH ROW  
BEGIN  
    IF INSERTING THEN  
        INSERT INTO emp_audit VALUES (:NEW.employee_id, 'INSERT', SYSDATE);  
    ELSIF UPDATING THEN  
        INSERT INTO emp_audit VALUES (:OLD.employee_id, 'UPDATE', SYSDATE);  
    ELSIF DELETING THEN  
        INSERT INTO emp_audit VALUES (:OLD.employee_id, 'DELETE', SYSDATE);  
    END IF;  
END;  
/  

```

Below the query editor, there is a "Script Output" window. It shows the execution results of the script:

```
Table EMP_AUDIT created.  
  
Trigger EMP_AUDIT_TR compiled
```

The "Script Output" window also includes a status bar indicating "Task completed in 0.196 seconds".

Worksheet Query Builder

```
CREATE OR REPLACE TRIGGER check_dept_fk
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM departments
    WHERE department_id = :NEW.department_id;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Invalid Department ID');
    END IF;
END;
/
```

Script Output x

Task completed in 0.041 seconds

Trigger CHECK_DEPT_FK compiled

Worksheet

Query Builder

```
ALTER TABLE employees ADD last_modified DATE;
```

```
CREATE OR REPLACE TRIGGER update_timestamp  
BEFORE UPDATE ON employees  
FOR EACH ROW  
BEGIN  
    :NEW.last_modified := SYSDATE;  
END;  
/
```

Script Output x

Task completed in 0.042 seconds

Table EMPLOYEES altered.

Trigger UPDATE_TIMESTAMP compiled

DDL Trigger Tasks:

Worksheet

Query Builder

```
CREATE TABLE ddl_audit (  
    ddl_date DATE,  
    user_name VARCHAR2(30),  
    object_type VARCHAR2(30),  
    object_name VARCHAR2(30),  
    operation VARCHAR2(30)  
);  
  
CREATE OR REPLACE TRIGGER ddl_audit_tr  
AFTER DDL ON SCHEMA  
BEGIN  
    INSERT INTO ddl_audit  
    VALUES (SYSDATE,  
            sys_context('USERENV', 'CURRENT_USER'),  
            ora_dict_obj_type,  
            ora_dict_obj_name,  
            ora_sysevent);  
END;  
/  

```

Script Output x

     | Task completed in 0.038 seconds

Table DDL_AUDIT created.

Trigger DDL_AUDIT_TR compiled

Worksheet Query Builder

```
CREATE OR REPLACE TRIGGER protect_table
BEFORE DROP OR ALTER ON SCHEMA
BEGIN
    IF ora_dict_obj_name = 'EMPLOYEES' THEN
        RAISE_APPLICATION_ERROR(-20002, 'Cannot modify EMPLOYEES table');
    END IF;
END;
/
```

Script Output x

Task completed in 0.035 seconds

Trigger PROTECT_TABLE compiled

Worksheet Query Builder

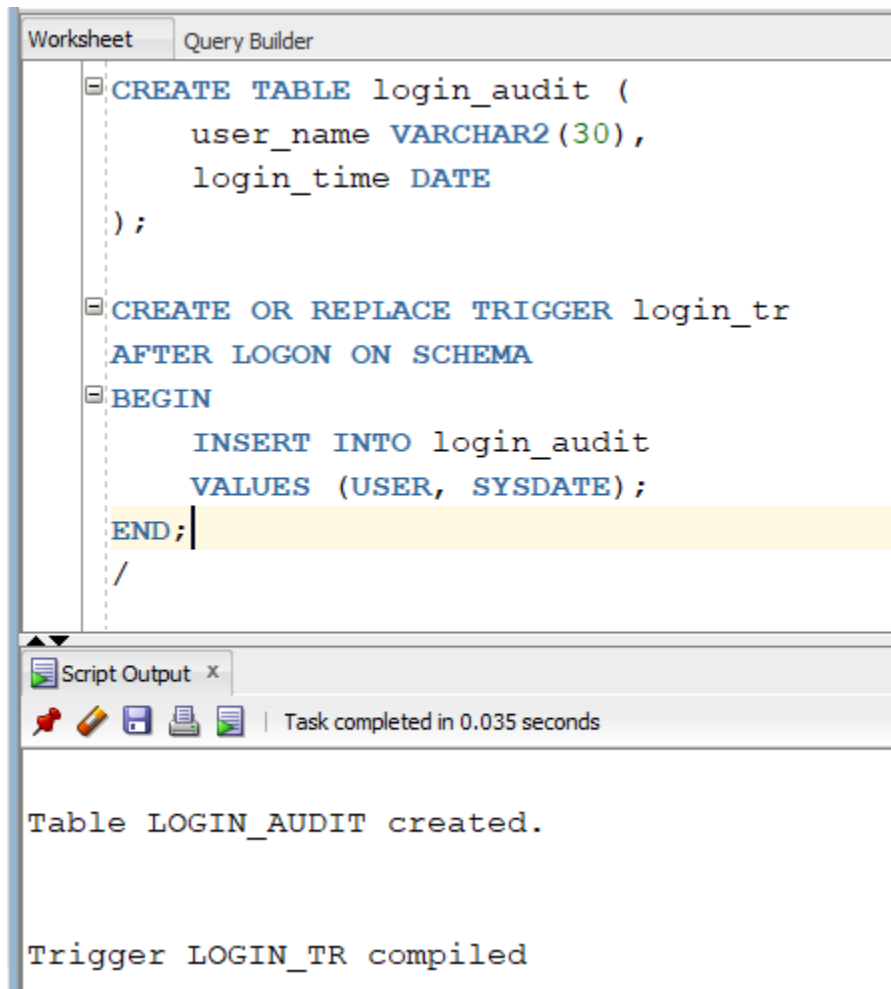
```
CREATE OR REPLACE TRIGGER prevent_table_creation
BEFORE CREATE ON SCHEMA
BEGIN
    IF ora_dict_obj_name LIKE 'TEST%' THEN
        RAISE_APPLICATION_ERROR(-20003, 'Table name not allowed');
    END IF;
END;
/
```

Script Output x

Task completed in 0.036 seconds

Trigger PREVENT_TABLE_CREATION compiled

System/Database Trigger Task:



The screenshot shows a database query builder interface with two tabs: "Worksheet" and "Query Builder". The "Query Builder" tab is active, displaying a SQL script. The script consists of two main parts: creating a table named "login_audit" and creating a trigger named "login_tr". The table creation part defines two columns: "user_name" of type "VARCHAR2(30)" and "login_time" of type "DATE". The trigger creation part is an "AFTER LOGON ON SCHEMA" trigger that inserts a record into the "login_audit" table with the values of "USER" and "SYSDATE". The script is as follows:

```
CREATE TABLE login_audit (  
    user_name VARCHAR2(30),  
    login_time DATE  
);  
  
CREATE OR REPLACE TRIGGER login_tr  
AFTER LOGON ON SCHEMA  
BEGIN  
    INSERT INTO login_audit  
    VALUES (USER, SYSDATE);  
END;  
/  
/
```

Below the script editor, there is a "Script Output" window. It contains a status bar that says "Task completed in 0.035 seconds". Below this, the output text reads:

```
Table LOGIN_AUDIT created.  
  
Trigger LOGIN_TR compiled
```

Worksheet Query Builder

```
CREATE OR REPLACE TRIGGER notify_dba
AFTER LOGON ON SCHEMA
BEGIN
    IF USER = 'HR' THEN
        DBMS_OUTPUT.PUT_LINE('Privileged user logged in!');
    END IF;
END;
/
```

Script Output x

Task completed in 0.035 seconds

Trigger NOTIFY_DBA compiled

Worksheet Query Builder

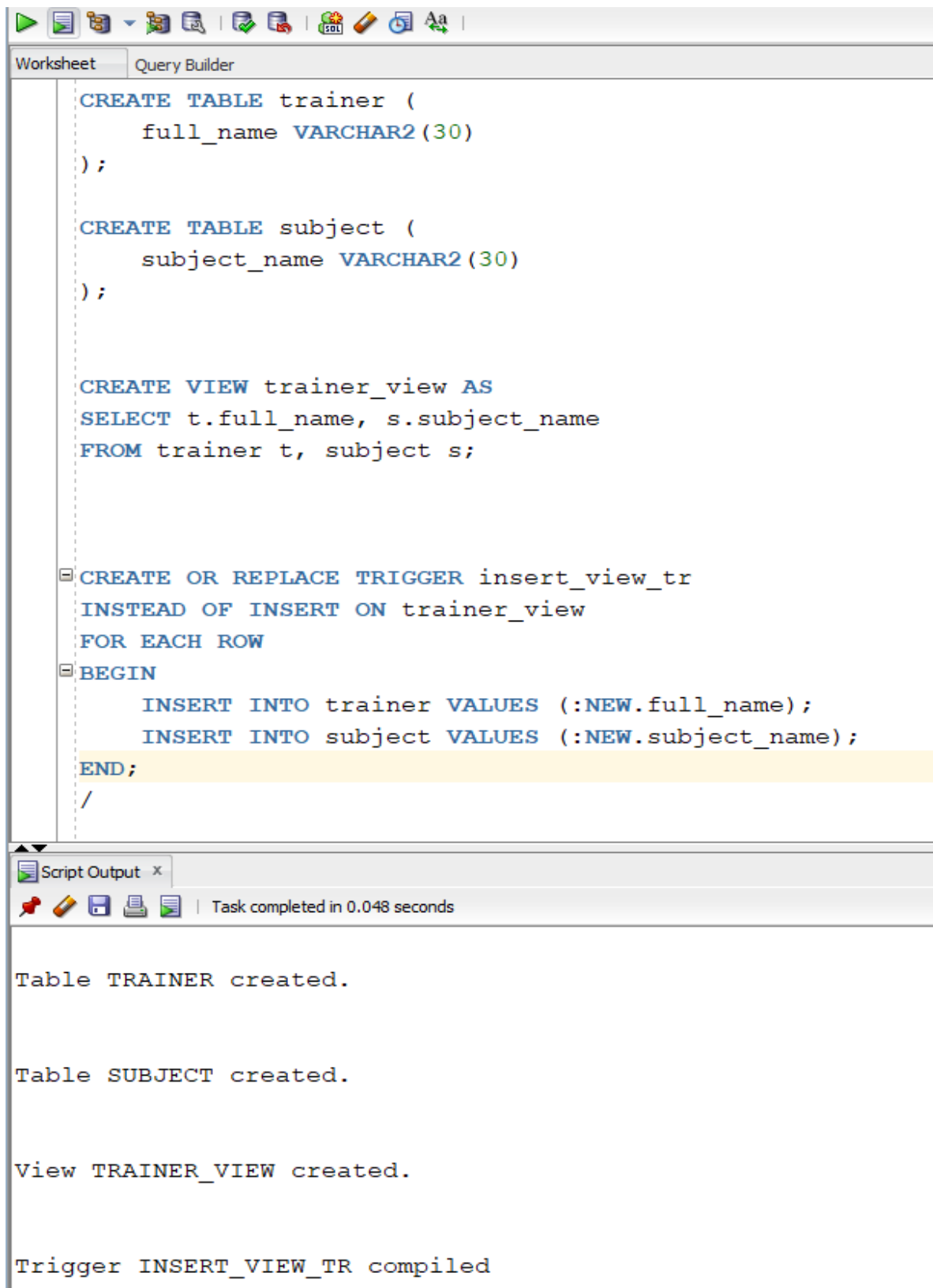
```
CREATE OR REPLACE TRIGGER set_timezone
AFTER LOGON ON SCHEMA
BEGIN
    EXECUTE IMMEDIATE 'ALTER SESSION SET TIME_ZONE = ''+05:00'';';
END;
/
```

Script Output x

Task completed in 0.046 seconds

Trigger SET_TIMEZONE compiled

Instead of Trigger Task:



The screenshot displays a database query editor interface. The top toolbar includes icons for running queries, saving, and other standard database operations. The main workspace is titled 'Query Builder' and contains the following SQL script:

```
CREATE TABLE trainer (  
    full_name VARCHAR2(30)  
);  
  
CREATE TABLE subject (  
    subject_name VARCHAR2(30)  
);  
  
CREATE VIEW trainer_view AS  
SELECT t.full_name, s.subject_name  
FROM trainer t, subject s;  
  
CREATE OR REPLACE TRIGGER insert_view_tr  
INSTEAD OF INSERT ON trainer_view  
FOR EACH ROW  
BEGIN  
    INSERT INTO trainer VALUES (:NEW.full_name);  
    INSERT INTO subject VALUES (:NEW.subject_name);  
END;  
/  

```

Below the query editor is a 'Script Output' window. It shows the results of the SQL execution:

```
Table TRAINER created.  
  
Table SUBJECT created.  
  
View TRAINER_VIEW created.  
  
Trigger INSERT_VIEW_TR compiled
```

Worksheet Query Builder

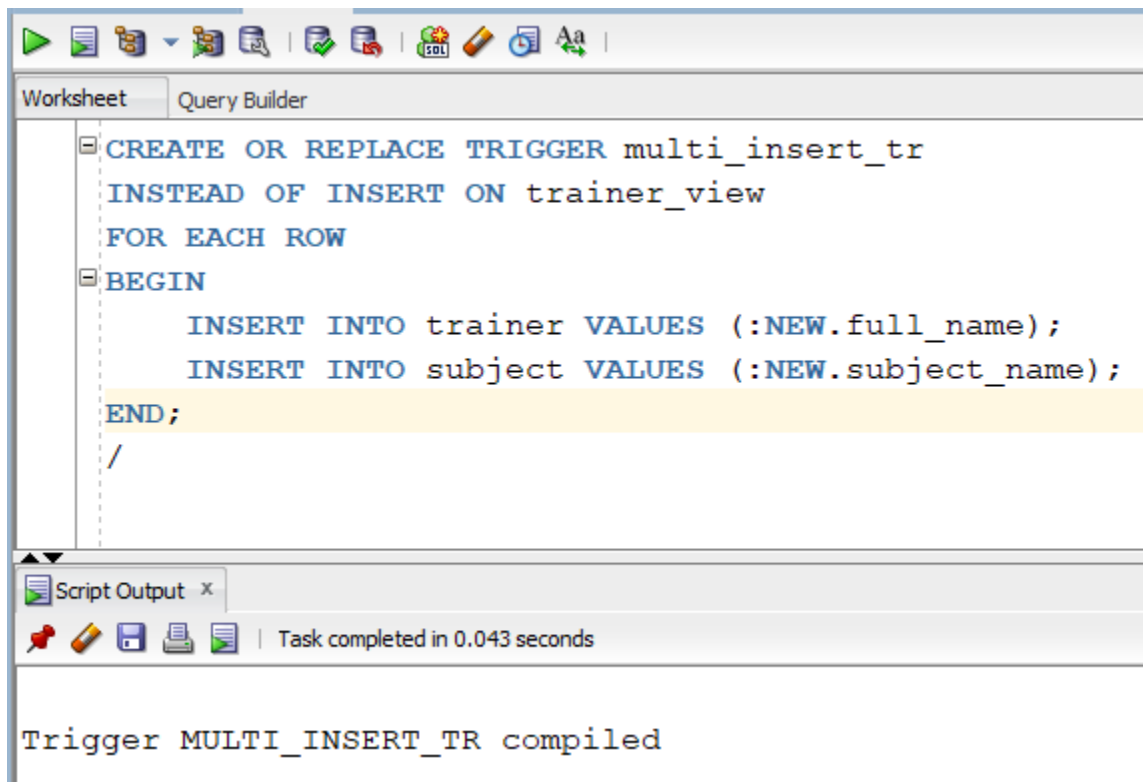
```
CREATE OR REPLACE TRIGGER update_view_tr
INSTEAD OF UPDATE ON trainer_view
FOR EACH ROW
BEGIN
    UPDATE trainer
    SET full_name = :NEW.full_name
    WHERE full_name = :OLD.full_name;

    UPDATE subject
    SET subject_name = :NEW.subject_name
    WHERE subject_name = :OLD.subject_name;
END;
/
```

Script Output x

     | Task completed in 0.039 seconds

```
Trigger UPDATE_VIEW_TR compiled
```



The screenshot shows a database query editor window. The top toolbar contains icons for running queries, saving, and other database operations. The main text area contains a SQL script to create or replace a trigger named 'multi_insert_tr' on the view 'trainer_view'. The script is as follows:

```
CREATE OR REPLACE TRIGGER multi_insert_tr
  INSTEAD OF INSERT ON trainer_view
  FOR EACH ROW
  BEGIN
    INSERT INTO trainer VALUES (:NEW.full_name);
    INSERT INTO subject VALUES (:NEW.subject_name);
  END;
```

The 'END;' line is highlighted in yellow. Below the script, there is a 'Script Output' tab with a message indicating the task was completed in 0.043 seconds.

Trigger MULTI_INSERT_TR compiled